

SQL and R – Movie Ratings

DATA 607: Week 2 Assignment: Approach

Bowen C

2026-09-09

Approach

The assignment is to collect ratings for six recent movies and store them in an SQL database. The way I approached this assignment is by fabricating the ratings (and the raters) and building up the required databases within SQL. I justify this approach since the actual ratings and the surveying methodology are not the focus of this assignment. I did, however, take a few liberties with the instructions. Instead of focusing on six movies, I asked my five “raters,” based on personas or film-goer types, to rate any six films they might have seen recently. The resulting movie database therefore contains more than six films. The other modification is that the ratings are on a 10-point scale. This was simply a result of the fact that I misread the instructions before I created the databases. Additionally, since my five raters obviously must have seen all six films that they rated, there will be no missing values for ratings. However, I will be sure to include instructions in the code for handling missing values, and I will also implement a secondary rating system using year-to-date box office gross, which does contain missing values for films that haven’t been publicly released yet.

Roadmap

I only covered one week of SQL in an Intro Comp Sci course I took a few years ago. I used W3Schools’ [PostgreSQL Tutorial](#) to walk me step-by-step through the process of creating a database and tables in PostgreSQL.

As per the Assignment instructions, I created 3 tables in SQL. One to store rater info (“users”), one to store movie ratings (“movies”), and one to store movie information (“movie_info”). I will then query the database in R and merge tables as necessary. The final goal is to calculate the average rating for each film and export that information back into the movie_info table’s avg_rating column in PostgreSQL. The avg_rating column is currently empty.

Code Base

Now I will make sure I can import the tables from PostgreSQL:

```
library(RPostgreSQL)
```

Loading required package: DBI

```
db607 <- dbConnect(  
  RPostgreSQL::PostgreSQL(),  
  dbname = "DATA 607",  
  host = "localhost",  
  user = "postgres",  
  password = ""  
)  
  
movies <- DBI::dbReadTable(db607, "movies")  
users <- DBI::dbReadTable(db607, "users")  
movie_info <- DBI::dbReadTable(db607, "movie_info")
```

Code Base

First let's merge the `movies` and `users` tables so we can see the names of the people who watched the films instead of just their User IDs. There is no practical reason for me to do this, but it's good coding practice.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.2.1      v readr      2.2.0  
v forcats    1.0.1      v stringr    1.6.0  
v ggplot2    4.0.3      v tibble     3.3.1  
v lubridate  1.9.5      v tidyr      1.3.2  
v purrr      1.2.2  
  
-- Conflicts ----- tidyverse_conflicts() --  
x dplyr::filter() masks stats::filter()  
x dplyr::lag()     masks stats::lag()  
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
# I accidentally added a zero in front of each User ID in the movies table, while the User ID
# was correct in the users table.

movies <- movies %>%
  mutate(user_id = as.character(as.integer(user_id)))

movies <- movies %>%
  left_join(users %>%
    select(user_id, name), by = "user_id")

movies <- movies %>%
  select(-user_id)
```

Now I will find the average rating for each film:

```
ratings <- movies %>%
  group_by(movie) %>%
  summarise(
    average_rating = mean(rating, na.rm = TRUE),
    number_ratings = sum(!is.na(rating)))
```

Now that I have the average ratings per user, I want to “upload” this info into the movie_info dataset.

```
# Convert zeros in ytd_gross to NA so I can handle NA values later:
# (The NAs are because the films haven't come out yet.)
movie_info <- movie_info %>%
  mutate(ytd_gross = na_if(ytd_gross, 0))

movie_info <- movie_info %>%
  left_join(ratings %>%
    select(movie, average_rating), by = c("title" = "movie"))
```

Replace NAs in avg_rating column with values from average_rating column and delete average_rating:

```
movie_info <- movie_info %>%
  mutate(avg_rating = average_rating) %>%
  select(-average_rating)
```

Conclusion:

At this point, I would want to actually upload the average rating data to the “live” database on Postgres. However, since we will probably be doing that in a later Assignment, I decided not to try doing that in this Assignment.

Here are the top ten films by average user rating:

```
top Rated <- movie_info[order(movie_info$avg_rating, decreasing = TRUE), ][1:10, ]

top Rated$rank <- seq_len(nrow(top Rated))

top Rated <- top Rated %>%
  select(rank, everything())

top Rated
```

	rank	title	year_out	avg_rating	ytd_gross
11	1	The Devil Wears Prada 2	2026	9.000000	220556651
12	2	Coyote vs. Acme	2026	9.000000	38604848
7	3	Backrooms	2026	8.000000	197523513
8	4	Primetime	2026	8.000000	NA
9	5	Spider-Man	2026	8.000000	923466178
10	6	Michael	2026	7.666667	372303388
4	7	Tony	2026	7.000000	13846159
13	8	Lesbian Space Princess	2025	7.000000	NA
14	9	The End of Oak Street	2026	7.000000	51717893
3	10	The Odyssey	2026	6.800000	588986925

And here are the top ten films by year-to-date box office gross:

```
top_grossing <- movie_info[order(movie_info$ytd_gross, decreasing = TRUE), ][1:10, ]

top_grossing$rank <- seq_len(nrow(top_grossing))

top_grossing <- top_grossing %>%
  select(rank, everything())

top_grossing
```

	rank	title	year_out	avg_rating	ytd_gross
--	------	-------	----------	------------	-----------

9	1	Spider-Man	2026	8.000000	923466178
3	2	The Odyssey	2026	6.800000	588986925
10	3	Michael	2026	7.666667	372303388
6	4	Obsession	2025	6.500000	263451715
11	5	The Devil Wears Prada 2	2026	9.000000	220556651
7	6	Backrooms	2026	8.000000	197523513
1	7	Disclosure Day	2026	6.000000	116636645
14	8	The End of Oak Street	2026	7.000000	51717893
12	9	Coyote vs. Acme	2026	9.000000	38604848
4	10	Tony	2026	7.000000	13846159

References

W3Schools. (no date). PostgreSQL Tutorial. <https://www.w3schools.com/postgresql/index.php>.